

Project Portfolio

1. Project Title

IoT-Based Air Monitoring System

2. Executive Summary

Air pollution is a major and growing threat to public health, and the World Health Organization (WHO) publishes concentration guidelines for key pollutant gases to help limit its impact. This project delivers a low-cost prototype Environmental Air Pollution Monitoring System that measures concentrations of harmful gases such as CO, CO₂, and NH₃ using inexpensive semiconductor gas sensors.

Sensor nodes built around an STM32 Nucleo-F446RE microcontroller collect environmental data and pass it to a Raspberry Pi 3B, which acts as the system's gateway/base station. The Raspberry Pi forwards the readings over Wi-Fi to ThingSpeak, an open-source IoT cloud platform, enabling near real-time monitoring and visualization of air quality from any internet-connected device. The result is a compact, affordable alternative to traditional air-quality monitoring equipment, suitable for deployment in homes, schools, colleges, and offices.

3. Problem Statement

According to the WHO, roughly 90% of people now breathe polluted air, and air pollution contributes to an estimated 7 million deaths every year; a further 1.5 million deaths annually are linked to indoor air pollution from solid-fuel cooking. Poor indoor air quality (IAQ) is associated with symptoms ranging from eye, nose, and throat irritation to headaches, dizziness, fatigue, and worsened asthma.

Conventional air-quality monitoring equipment is often expensive and impractical for widespread, everyday deployment in homes, schools, and offices. There is a clear need for a low-cost, IoT-enabled system that can continuously sense key pollutant gases and make the data available remotely and in real time, so that occupants and administrators can identify and respond to poor air quality quickly.

4. Objectives

1. Design a low-cost prototype system to monitor concentrations of major indoor air pollutant gases (CO, CO₂, NH₃, and related gases).
2. Interface low-cost semiconductor gas sensors (MQ135 and MQ7) with an STM32 Nucleo-F446RE microcontroller node.
3. Use a Raspberry Pi 3B as a gateway/base station to collect sensor data from the microcontroller node over GPIO.
4. Transmit sensor data from the Raspberry Pi to the ThingSpeak cloud platform for storage, visualization, and remote real-time monitoring.

5. Develop a simple, low-overhead calibration technique to translate raw sensor output into indicative pollution-severity levels.

5. System Architecture

The system follows standard three-layer IoT architecture:

- Perception (sensing) layer: MQ135 and MQ7 semiconductor gas sensors mounted on a breadboard and wired to the Nucleo-F446RE, which forms the sensing "node."
- Network layer: The Raspberry Pi 3B connects to the internet via its built-in Wi-Fi and transmits sensor readings to the ThingSpeak cloud platform approximately every 15 seconds.
- Application/presentation layer: A Raspberry Pi base station handles data received from the node and forwards it to ThingSpeak, which performs data storage and visualization so readings can be viewed as live charts from any browser.

Overall data flow: Gas sensors → Nucleo-F446RE (signal conditioning and range classification) → GPIO link → Raspberry Pi 3B (gateway) → Wi-Fi → ThingSpeak (cloud storage and visualization).

6. Hardware Architecture

Microcontroller

STM32F446RET6 Nucleo-64 board — a 32-bit ARM Cortex-M4 CPU running up to 180 MHz, with 512 KB Flash, three 12-bit ADCs across 24 channels, two 12-bit DACs, Arduino-compatible digital I/O and analog input pins, and USB connectivity for programming.

Sensors

MQ135 (detects NH₃, CO₂, and other common air-quality gases) and MQ7 (a carbon-monoxide sensor). Both use a tin-dioxide (SnO₂) sensing element whose conductivity rises with increasing target-gas concentration; a simple analog output signal is read directly by the Nucleo board's ADC.

Gateway

Raspberry Pi 3B — a credit-card-sized computer running Raspbian Linux, with a quad-core 64-bit ARM Cortex-A53 CPU at 1.2 GHz, 1 GB SDRAM, and built-in Wi-Fi/Bluetooth. It receives sensor status over GPIO from the Nucleo board and pushes it to ThingSpeak, and is accessed remotely over WLAN via VNC Viewer.

Power Supply

The MQ135 sensor is powered with 5V DC supplied from the Nucleo-F446RE board, while the MQ7 sensor is powered with 5V DC from the Raspberry Pi. All board and sensor ground pins are tied to a common ground.

Reasons for Selection

- Low-cost semiconductor gas sensors keep the overall bill of materials affordable, making the system practical for widespread deployment.
- The Nucleo-F446RE's ARM Cortex-M4 core, multiple ADC channels, and Arduino-compatible I/O made it well suited to reading two analog gas sensors and performing lightweight on-board classification.
- The Raspberry Pi 3B's built-in Wi-Fi and general-purpose Linux environment made it an efficient, low-complexity gateway for internet connectivity and remote access, without requiring a dedicated monitor.
- ThingSpeak was selected as an open-source IoT analytics platform that supports live data visualization out of the box, without needing custom dashboard code.

7. Circuit Diagram

The MQ135 and MQ7 sensors are wired to the Nucleo-F446RE board, which drives four digital output pins (PA11, PA13, PB1, PB2) to signal each sensor's calibrated range band to the Raspberry Pi's GPIO inputs (GPIO 5, GPIO 6, GPIO 23, GPIO 24). The Nucleo board supplies 5V to the MQ135 sensor, and the Raspberry Pi supplies 5V to the MQ7 sensor; all grounds are tied together.

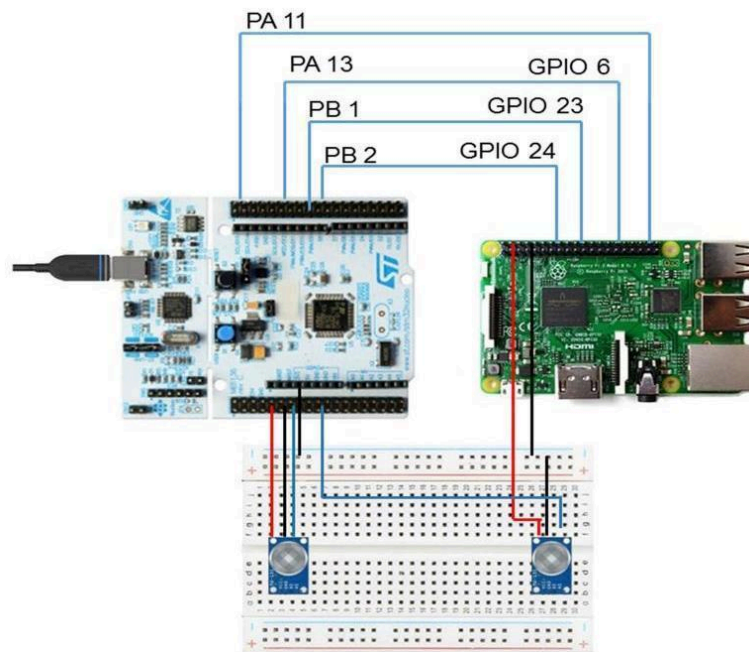


Figure 1: Schematic for sensor interface with boards — Nucleo-F446RE, MQ sensors, and Raspberry Pi 3B

8. Firmware Architecture

The firmware running on the STM32F446RE Nucleo board was developed in Embedded C using STM32CubeMX for pin/peripheral configuration and Keil IDE for coding, building, and flashing. At runtime, the firmware:

1. Reads the analog output of the MQ135 and MQ7 sensors via the board's on-chip ADC.
2. Compares each reading against the calibrated range to classify it into a low/medium/high severity band.
3. Drives the corresponding GPIO output pin (PA11/PA13 for MQ135, PB1/PB2 for MQ7) high to signal the classified band to the Raspberry Pi.

9. Software Architecture

On the Raspberry Pi 3B, a Python application reads the GPIO pin states set by the Nucleo board and maps each combination back to a representative sensor value (75/150/255 for MQ135; 50/100/255 for MQ7). It then publishes these values as separate fields on ThingSpeak using the platform's write API, at an interval of roughly 15 seconds. The Pi runs Raspbian Linux and is administered remotely over Wi-Fi using VNC Viewer, so no monitor or keyboard is required at the deployment site.

10. Communication Flow

1. Gas sensors output an analog voltage proportional to detected gas concentration.
2. The Nucleo-F446RE's ADC samples this voltage (configured to 8-bit resolution) and classifies it into a range band.
3. The Nucleo board sets the matching digital GPIO pin high, signaling the band to the Raspberry Pi over a direct GPIO link.
4. The Raspberry Pi's Python script polls these GPIO inputs, resolves them to representative sensor values, and pushes the values to ThingSpeak over the Pi's built-in Wi-Fi connection, roughly every 15 seconds.
5. ThingSpeak stores the incoming values and renders them as live field charts, viewable in near real time from any browser.

11. Hardware Implementation

The prototype was assembled using the following components:

- STM32F446RET6 Nucleo board
- Raspberry Pi 3 Model B
- MQ-135 sensor
- MQ-7 sensor
- Jumper wires
- Breadboard
- D-Link router

The MQ135 and MQ7 sensors were mounted on a breadboard and connected to the Nucleo-F446RE board to sense ambient pollution levels. The Nucleo board supplied 5V DC to the MQ135 sensor, while the Raspberry Pi supplied 5V DC to the MQ7 sensor. GPIO pins were used to link the Raspberry Pi and the Nucleo board, and all component and board ground pins were connected to a common ground, per the wiring shown in Figure 2.

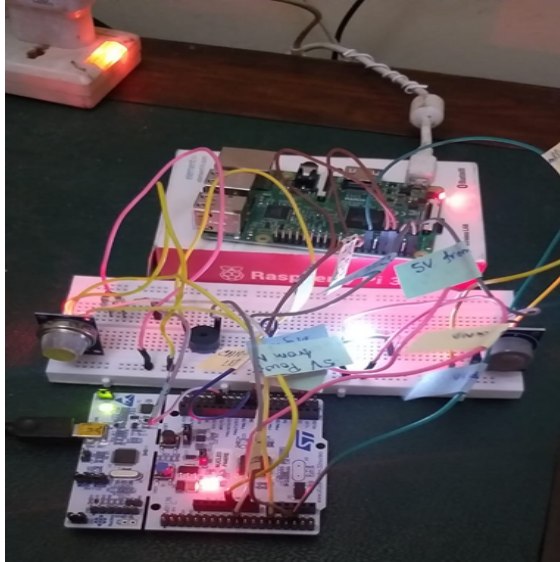


Figure 2: Hardware setup

12. Testing & Validation

The customized sensor calibration technique was validated by configuring the ADC to 8-bit resolution (0–255 range) and segmenting expected output into three bands per sensor:

- MQ135: Range 0–75, Range 76–150, Range 151–255 (values taken: 75, 150, 255).
- MQ7: Range 0–50, Range 51–100, Range 101–255 (values taken: 50, 100, 255).

For each test, the maximum value within the sensor's active range band was reported, and the corresponding GPIO signaling logic was exercised to confirm that band transitions correctly propagated from sensor reading through to the value transmitted to ThingSpeak. Observations confirmed that increasing gas concentration produced the expected step-wise increase in transmitted value, and that the ThingSpeak field charts updated accordingly at the configured ~15-second interval.

13. Results

The system successfully transmitted classified MQ135 and MQ7 readings to ThingSpeak, where they were plotted as live field charts ("Air Monitoring System"). The captured results show the MQ135 field trending upward from roughly 75 toward 150 over the observation window, while the MQ7 field trended downward from roughly 255 toward 50, consistent with the expected step-wise range classification described in Section 12.

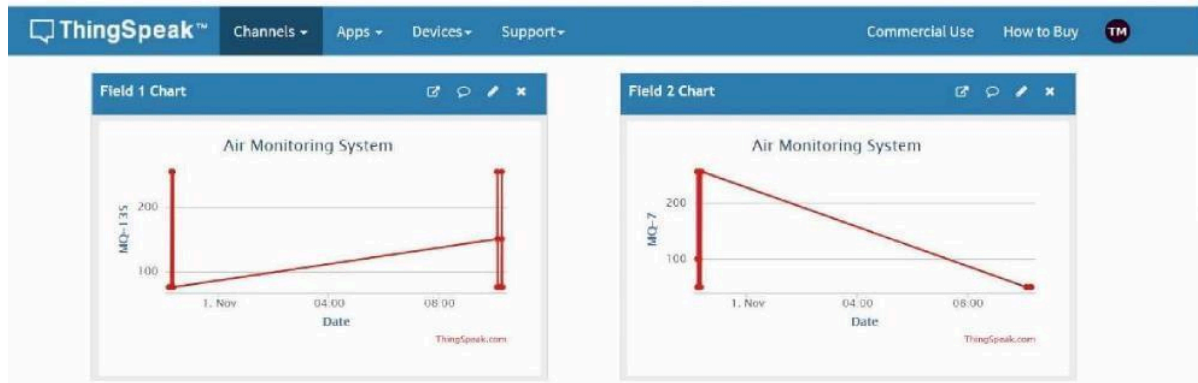


Figure 3: MQ135 (Field 1) and MQ7 (Field 2) sensor data as received and charted on ThingSpeak

14. My Contribution

As the developer of this prototype, my contributions spanned the full stack of the system:

- Selected and specified the hardware components (Nucleo-F446RE, Raspberry Pi 3B, MQ135, MQ7) and designed the overall three-layer IoT architecture.
- Wired and assembled the sensor node on a breadboard, including power and ground distribution between the Nucleo board and the Raspberry Pi.
- Developed the Embedded C firmware for the Nucleo-F446RE (using STM32CubeMX and Keil IDE) to sample sensor ADC values and classify them into calibrated range bands.
- Wrote the Python application on the Raspberry Pi to read GPIO signals from the Nucleo board and publish resolved sensor values to ThingSpeak.
- Designed and validated the customized sensor calibration/range-classification technique described in Section 12.
- Configured remote access to the Raspberry Pi via VNC over Wi-Fi and set up the ThingSpeak channel and field charts used for real-time monitoring.

15. Engineering Challenges

Challenge	Solution
Raw analog output from the low-cost MQ135/MQ7 sensor. It is noisy and lacks a direct PPM conversion.	Configured the ADC to 8-bit resolution and applied a customized three-band approximation (described in section 12) to classify readings into indicative severity ranges instead of relying on unverified PPM formulas.
Limited pins available to pass calibrated sensor status from the Nucleo board to the Raspberry Pi.	Encoded each sensor's range band as a digital high/low signal on two dedicated GPIO pins per sensor, keeping the Nucleo-to-Pi interface simple and reliable.
Needed remote access to the Raspberry Pi gateway without a dedicated monitor.	Configured the Pi's built-in Wi-Fi with VNC Viewer for headless remote access and control from a laptop over WLAN.

Ensuring near real-time visibility of pollutant levels for a remote, non-technical viewer.	Pushed calibrated sensor values from the Pi to ThingSpeak at 15-second intervals, using its built-in charting so data could be viewed live from any browser without extra code.
--	---

16. Skills Demonstrated

- Embedded systems programming in Embedded C on an ARM Cortex-M4 (STM32CubeMX, Keil IDE).
- Analog sensor interfacing and ADC-based signal classification.
- Single-board computer (Raspberry Pi) configuration, GPIO programming, and Python scripting.
- IoT system design across perception, network, and application layers.
- Cloud data integration and visualization using ThingSpeak.
- Circuit design, breadboard prototyping, and hardware debugging.
- Remote systems administration (Wi-Fi, VNC) for headless device access.
- Data calibration technique design for low-cost sensor hardware.

17. Future Enhancements

- Add dedicated PM2.5/PM10 particulate-matter sensors for more accurate, comprehensive air-quality measurement.
- Replace the coarse three-band GPIO signaling scheme with finer-grained ranges and a proper PPM conversion for more precise readings.
- Scale to multiple hardware modules for coverage of larger spaces.
- Adopt ZigBee to improving range and network scalability for multi-node deployments.

18. Lessons Learned

Low-cost semiconductor gas sensors are a viable foundation for real-time air-quality monitoring: despite their simplicity, they offer quick response, low maintenance, continuous measurement, and a small form factor suitable for compact IoT nodes. At the same time, translating their raw analog output into meaningful, trustworthy readings requires a deliberate calibration strategy — the project's range-banding approach was an effective, low-complexity way to get indicative readings working end-to-end, but it also highlighted the trade-off between simplicity and measurement precision, and the value of a clear GPIO-based protocol when working within a limited number of I/O pins between the microcontroller node and the gateway.